

Homework 7

Exercise 1 Hessian of functions

Feature 1

1. Determine the Hessians of the functions defined on page 4 and implement them.
2. Write a function whose input is a symmetric matrix, and such that:
 - the function computes the eigenvalues of the matrix. You can use the function `numpy.linalg.eigvalsh()` to compute the eigenvalues.
 - the function returns the highest and lowest eigenvalues of the matrix.
3. Write a function which receives the highest and lowest eigenvalues λ_n and λ_1 of a symmetric matrix. The function prints λ_1 , λ_n , and a message according to the following conditions:
 - If $\lambda_1 > 0$, print **Definite positive matrix**.
 - If $\lambda_n < 0$, print **Definite negative matrix**.
 - If $\lambda_1 < 0$ and $\lambda_n > 0$, print **Indefinite matrix**.

Solution

Example 2

See `sample_functions.py` and `p1.py`.

Exercise 2 Joined gradient descent and Newton method

Feature 3

The function to implement receives the following parameters:

- The objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$,
- The gradient ∇f of the objective function,
- The hessian $\nabla^2 f$ of the objective function,
- The initial point x_0 ,
- The maximum number of iterations N ,
- A threshold $\tau > 0$,
- The parameters α_{init} , c_1 , ρ , and a maximum number of iterations N_b for the backtracking algorithm.

The function to implement performs the following steps: Fix $m = 0$. For $k = 0, \dots, N - 1$, do the following steps:

1. Compute the gradient g_k at the point x_k , i.e. $g_k = \nabla f(x_k)$.
2. If $\|g_k\| < \tau$, do $\text{res} = 1$ and stop the loop.
3. If the stopping criterion is not fulfilled, compute the Hessian at x_k : $B_k = \nabla^2 f(x_k)$.
4. Try to perform a Cholesky decomposition of B_k :
 - if the decomposition holds, we obtain $B_k = LL^\top$. Then, solve the system $LL^\top p_k = -g_k$ to obtain the descent direction p_k .
 - If the factorization fails, define the descent direction as $p_k = -g_k$, then do $m = m + 1$.
5. Compute the step size α_k using the backtracking Algorithm with the condition of sufficient descent.
6. Compute the updating of the sequence $x_{k+1} = x_k + \alpha_k p_k$.

Return x_k , g_k , m and res .

Solution

Example 4

See `p2.py` and `backtracking.py`.

Exercise 3 Strongly convex functions and quasi-Newton methods

Feature 5

1. Show that if $g \in C^1(\mathbb{R}^n)$ is convex, then

$$(\nabla g(x) - \nabla g(y))^\top (x - y) \geq 0 \quad \forall x, y \in \mathbb{R}^n. \quad (1)$$

2. Deduce from 1. that a strongly convex function $f \in C^1(\mathbb{R}^n)$ of parameter $m > 0$ satisfies that

$$(\nabla f(x) - \nabla f(y))^\top (x - y) \geq m\|x - y\|^2 \quad \forall x, y \in \mathbb{R}^n \quad (2)$$

3. Deduce from 2. a property of quasi-Newton methods applied to strongly convex functions.

Solution

Example 6

1. The first order condition for convexity states

$$\begin{aligned} g(y) &\geq g(x) + \nabla g(x)^\top (y - x), \\ g(x) &\geq g(y) + \nabla g(y)^\top (x - y) \end{aligned} \quad (3)$$

Thus, adding these inequalities gives

$$\begin{aligned} g(x) + g(y) &\geq g(x) + g(y) + (\nabla g(y) - \nabla g(x))^\top (x - y) \\ &\Leftrightarrow (\nabla g(x) - \nabla g(y))^\top (x - y) \geq 0. \end{aligned} \quad (4)$$

2. By definition,

$$\begin{aligned} f(y) &\geq f(x) + \nabla f(x)^\top (y - x) + \frac{m}{2}\|y - x\|^2, \\ f(x) &\geq f(y) + \nabla f(y)^\top (x - y) + \frac{m}{2}\|x - y\|^2 \end{aligned} \quad (5)$$

and thus, adding both inequalities gives the desired inequality:

$$(\nabla f(x) - \nabla f(y))^\top (x - y) \geq m\|x - y\|^2 \quad (6)$$

3. The curvature is strictly positive ($s_k^\top y_k > 0$).

Algorithm 1: Modified BFGS

```
1 Input  $f(x), \nabla f(x), x_0, \tau > 0, H_0$  an approximation of the inverse Hessian of  $f$  at  $x_0$ ,  $N$  the  
   maximum number of iterations, and the parameters for the backtracking algorithm.  
2 Compute  $g_0 = \nabla f(x_0)$   
3 Define  $I$  as the identity matrix and  $\text{res} = 0$   
4 for  $k = 0, 1, \dots, N - 1$  do  
5   if  $\|g_k\| < \tau$  then  
6     Set  $\text{res} = 1$   
7     break  
8   end  
9   Compute  $p_k = -H_k g_k$   
10  if  $p_k^\top g_k > 0$  then  
11     $\lambda_1 = 10^{-5} + \frac{p_k^\top g_k}{g_k^\top g_k}$   
12     $H_k = H_k + \lambda_1 I$   
13     $p_k = p_k - \lambda_1 g_k$   
14  end  
15   $x_{k+1} = x_k + \alpha p_k$ , for  $\alpha_k$  obtained from backtracking  
16   $s_k = x_{k+1} - x_k$   
17   $y_k = g_{k+1} - g_k$   
18  if  $y_k^\top s_k \leq 0$  then  
19     $\lambda_2 = 10^{-5} - \frac{y_k^\top s_k}{y_k^\top y_k}$   
20     $H_{k+1} = H_k + \lambda_2 I$   
21  else  
22     $\rho_k = \frac{1}{y_k^\top s_k}$   
23     $H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top$   
24  end  
25 end  
26 return  $x_k, g_k, k, \text{res}$   
27 end
```

Exercise 4 Modified BFGS Algorithm

Feature 7

1. Implement the modified BFGS algorithm. If the objective function has 2 variables, make the algorithm return the list of all the points $x_0, \dots, x_k$ to be able to use them later on to plot the trajectory.
2. Test the algorithm on the functions and initial points x_0 defined on page 4, where:
 - H_0 is the identity.
 - The maximum number of iterations is $N = 10000$.
 - The threshold $\tau = \sqrt{n\varepsilon_m}$, where ε_m is the machine epsilon.
 - For the backtracking algorithm, use $\alpha_{\text{init}} = 1, c_1 = 0.1, \rho = 0.6$, and the maximum number of iterations is $N_b = 100$.

Print $f(x_0)$ and the following values at each iteration:

- the iteration number k .
- the norm of the gradient $\|g_k\|$.
- if $n \leq 6$, print the vector x_k . Otherwise, print the first and last three components of x_k .
- $f(x_k)$

If $n = 2$, plot the level lines of the function and the trajectory defined by all the points $x_0, \dots, x_k$ generated by the algorithm.

Solution

Example 8

See p4.py.





